

# OutfitLock AI — Internship Assignment Submission

---

## 1. Project Overview

---

OutfitLock AI is an AI-powered fashion image generation tool that generates realistic fashion outputs while preserving outfit consistency from uploaded garment images.

The application allows users to:

- Upload outfit images
- Upload reference images
- Generate multiple AI fashion outputs
- Maintain garment consistency
- Download generated outputs
- Handle multiple outfit uploads and ZIP uploads

The system uses:

- React + TypeScript frontend
  - Django REST backend
  - Google Gemini Vision for garment analysis
  - Google Imagen (Nano Banana / Vertex AI) for image generation
- 

## 2. Tech Stack

---

### Frontend

---

- React
- TypeScript
- Tailwind CSS
- Vite
- Axios

### Backend

---

- Django
- Django REST Framework
- Python

### AI Services

---

- Google Gemini 2.5 Flash
- Google Vertex AI Imagen

### Cloud Services

---

- Google Cloud Platform
  - Vertex AI
- 

## 3. Working Features

---

### Implemented Features

---

#### Outfit Upload

Users can:

- Upload single outfit images
- Upload multiple outfit images
- Upload ZIP files containing outfit images

#### Reference Upload

Users can upload:

- Pose references
- Style references

- Background references
- Lighting references

## AI Fashion Generation

The tool:

- Analyzes uploaded outfit images
- Extracts garment information dynamically
- Builds preservation-focused prompts
- Generates multiple realistic fashion outputs

## Output Gallery

Generated images are displayed in:

- Live output gallery
- Download-ready format
- Multiple generated variations

## Consistency Preservation

The system preserves:

- Garment color
- Garment structure
- Sleeve type
- Fit
- Collar
- Texture appearance
- Outfit category

---

# 4. System Architecture

## AI Generation Pipeline

```
Frontend Upload
↓
Django Backend
↓
Gemini Vision Garment Analysis
↓
Dynamic Prompt Construction
↓
Vertex AI Imagen Generation
↓
Generated Outputs Returned
↓
Frontend Gallery Rendering
```

---

# 5. How Outfit Consistency Is Maintained

Outfit consistency is maintained using a multi-stage pipeline.

## Step 1 — Garment Analysis

Uploaded outfit images are analyzed using Gemini Vision.

The analyzer extracts:

- Garment type
- Color
- Sleeve style
- Fabric appearance
- Collar shape
- Fit structure
- Fashion category

## Step 2 — Dynamic Prompt Construction

---

The extracted garment details are injected into a strict preservation prompt.

The prompt enforces:

- Same color preservation
- Same garment structure
- Same sleeve length
- Same fit and silhouette
- Same texture appearance
- Same outfit category

## Step 3 — Imagen Generation

---

Google Imagen generates fashion outputs using the preservation prompt.

Additional prompt constraints prevent:

- Color drift
- Design drift
- Random garment changes
- Texture modifications

---

## 6. Nano Banana 2 / Imagen Integration

---

The project integrates Google Vertex AI Imagen for fashion image generation.

### Integration Steps

---

#### Step 1 — Google Cloud Setup

- Create Google Cloud project
- Enable Vertex AI API
- Configure billing
- Install Google Cloud SDK
- Authenticate using gcloud CLI

#### Step 2 — Vertex AI Initialization

The backend initializes Vertex AI using:

- Project ID
- Region
- Vertex AI SDK

#### Step 3 — Imagen Model Usage

The application uses Imagen generation models through:

```
ImageGenerationModel.from_pretrained()
```

The model receives:

- Dynamic garment prompts
- Fashion consistency instructions
- Editorial photography styling

#### Step 4 — Output Generation

Generated images are:

- Saved to media/outputs/
- Returned through Django API
- Rendered inside frontend output gallery

---

## 7. Local Setup Instructions

---

### Backend Setup

---

---

## Create Virtual Environment

```
python -m venv venv
```

## Activate Environment

```
venv\Scripts\activate
```

## Install Dependencies

```
pip install -r requirements.txt
```

## Configure Environment Variables

Create `.env` inside backend:

```
SECRET_KEY=django-insecure-outfitlock-ai-secret-key
DEBUG=True
GEMINI_API_KEY=YOUR_GEMINI_KEY
GOOGLE_API_KEY=YOUR_GOOGLE_KEY
VERTEX_API_KEY=YOUR_VERTEX_KEY
```

## Run Django Server

```
python manage.py runserver
```

---

# Frontend Setup

---

## Install Dependencies

```
npm install
```

## Run Frontend

```
npm run dev
```

---

# 8. Google Cloud Console Setup

---

## Enable APIs

---

Enable:

- Vertex AI API
- Generative AI API

## Authenticate

---

Install Google Cloud SDK and run:

```
gcloud init
```

Then:

```
gcloud auth application-default login
```

## Set Project

---

```
gcloud config set project YOUR_PROJECT_ID
```

---

## 9. Sample Inputs Used

### Outfit Images

---

Examples used:

- Plain green t-shirt
- Fashion tops
- Casual apparel

### Reference Images

---

Examples used:

- Model poses
- Fashion photography references
- Lighting references

---

## 10. Sample Outputs

The system generates:

- Fashion model outputs
- Ecommerce-style photography
- Outfit-preserving generations
- Multiple image variations

---

## 11. Known Limitations

Current limitations:

- Imagen is not a dedicated virtual try-on model
- Exact stitching preservation is not always perfect
- Some generations may slightly drift in pose or framing
- Complex printed garments may require stronger conditioning
- Batch generation increases generation time

---

## 12. Future Improvements

Possible improvements:

- Dedicated virtual try-on model integration
- Better reference conditioning
- Human parsing and segmentation
- Background replacement system
- Bulk ZIP output download
- Output history database
- Authentication and user accounts
- Cloud deployment optimization
- Advanced garment masking
- Improved consistency scoring

---

## 13. Assumptions Made During Development

- 
- Users provide clean outfit product images
  - Uploaded garments are clearly visible
  - Reference images guide styling but should not replace outfit identity
  - Internet access is available for AI services
  - Google Cloud APIs remain enabled and authenticated
- 

## 14. Repository Structure

---

```
frontend/  
backend/  
media/  
  uploads/  
  outputs/  
  references/
```

---

## 15. Deployment Notes

---

The project can be deployed using:

- Frontend: Netlify / Vercel
- Backend: Railway / Render / GCP VM

Environment variables must be configured during deployment.

---

## 16. Conclusion

---

OutfitLock AI successfully demonstrates an AI-powered outfit-consistent fashion image generation workflow using Gemini Vision and Google Imagen.

The project supports:

- Dynamic garment analysis
- AI-driven fashion generation
- Multiple outputs
- Outfit consistency preservation
- Modern frontend UX
- Scalable backend architecture

The system aligns closely with the assignment requirements and demonstrates practical integration of multimodal AI pipelines for fashion generation workflows.